

PROYECTO FINAL ANÁLISIS Y DISEÑO DE ALGORITMOS

**María José Castañeda Grajales
Kevin Santiago Lopez Salazar**

Luz Enith Guerrero Mendieta

**Universidad de Caldas
Facultad de inteligencia artificial e ingenierías
Manizales, Caldas
2025**

Índice

Índice.....	2
1. Descripción general del proyecto.....	3
2. Fundamentos teóricos.....	3
2.1. Representación geométrica.....	3
2.2. Tensores de probabilidad condicional.....	4
2.3. Función de costo.....	4
3. Estructura del código.....	5
3.1. Clase geometric.py – Secuencial.....	5
3.2. Clase geometric_mp.py – Paralelo.....	15
3.3. Clase geometric_cuda.py – Paralelo.....	21
4. Pruebas y resultados.....	24
4.1. Discrepancia de pérdida.....	24
4.2. Speedup promedio.....	25
4.3. Comparativa Estrategias (20N).....	25
4.4. Comparativa Estrategias (21N).....	27
4.5. Comparativa Estrategias (21N) - Tiempos por función.....	28
4.6. Uso de recursos del sistema.....	29
• Geometric – versión secuencial.....	29
• Geometric + MP.....	30
• Geometric + MP + CUDA.....	31
5. Conclusiones.....	34

1. Descripción general del proyecto

Este proyecto implementa una estrategia de análisis geométrico basada en el modelo SIA, con el fin de encontrar e identificar biparticiones óptimas en un sistema dinámico de variables binarias. Esta estrategia surge como una respuesta a las limitaciones encontradas en los enfoques tradicionales de análisis de sistemas complejos, los cuales suelen ser computacionalmente costosos, poco escalables y difíciles de interpretar cuando se trabaja con un número un poco elevado de variables interrelacionadas. El principal objetivo es encontrar una forma eficiente de dividir los subsistemas en partes que conserven la mayor cantidad de relaciones causales entre las variables sin necesidad de reconstruir o recalculer toda la estructura del sistema en cada iteración.

Para lograr esto, Se propone una representación geométrica del sistema como un cubo n -dimensional, en donde cada uno de los vértices representa un posible estado del sistema y cada arista representa una transición entre los estados que difieren en una sola variable, es decir, tiene correspondencia biyectiva.

Ese enfoque se fundamenta en el uso de herramientas topológicas y algebraicas como los tensores de probabilidad condicional y la distancia de Hamming, las cuales permiten calcular una función de coste asociada a las transiciones entre estados, basada en la diferencia probabilística entre configuraciones y en la cercanía geométrica de los estados. De esta forma el algoritmo puede explorar de forma eficiente las posibles biparticiones y seleccionar la que minimice la pérdida de información.

2. Fundamentos teóricos

En esta sección se abordan los conceptos matemáticos y computacionales que sustentan la estrategia geométrica del modelo. El propósito es proporcionar una base sólida que permita comprender por qué se utiliza la representación del sistema como un hipercubo, cómo se derivan las funciones de costo y de qué manera los tensores condicionales permiten capturar las dependencias dinámicas entre las variables.

2.1. Representación geométrica

Esta representación se basa en mapear cada configuración binaria posible del sistema como un vértice en un hipercubo n -dimensional, donde n es el número de variables. Esta estructura geométrica permite visualizar y analizar las transiciones del sistema como movimientos entre vértices vecinos en el cubo. Dos estados están conectados si difieren en exactamente una variable (un solo bit), y esa relación es lo que define la métrica de distancia Hamming. Esta representación es clave porque

permite aprovechar propiedades topológicas del hipercubo para organizar los cálculos y reducir la complejidad del análisis causal del sistema.

2.2. Tensores de probabilidad condicional

Un tensor condicional es una estructura que describe, para cada variable futura, cómo cambia su valor dependiendo del estado actual completo del sistema. Para cada variable se construye un tensor que contiene la probabilidad de que esa variable adopte los valores 0 o 1 en el siguiente instante de tiempo, condicionado al estado actual del sistema. Estos tensores se generan para cada variable por separado bajo el supuesto de independencia condicional: cada variable futura depende únicamente del estado presente y no directamente de otras variables futuras.

2.3. Función de costo

La función de costo $t(i, j)$ cuantifica la influencia causal de una transición entre dos estados i y j del sistema. Esta función considera tres elementos fundamentales: (1) la distancia de Hamming entre los estados, que se traduce en el número de variables distintas; (2) la diferencia absoluta entre los valores del tensor en esos estados, lo cual refleja cuánto varía la probabilidad de cambio de la variable analizada; y (3) una suma ponderada de los costos intermedios desde estados vecinos, para capturar el efecto acumulado de rutas alternativas. El factor $\gamma = 2^{-d(i,j)}$ introduce una penalización natural para transiciones lejanas, dándoles menor peso en el cálculo del costo total. Esta función permite comparar la "dificultad" o el impacto de diferentes trayectorias dentro del hipercubo, y sirve como núcleo para evaluar la calidad de distintas biparticiones.

Se define como:

$$t_x(i, j) = \gamma \cdot \left(|X[i] - X[j]| + \sum_{k \in N(i,j)} \{t(k, j)\} \right)$$

donde:

- $\gamma = 2^{-d(i,j)}$: factor de decaimiento exponencial según la distancia Hamming.

3. Estructura del código

3.1. Clase `geometric.py` – Secuencial

```
1 def descomponer_en_tensoros(self):
2     tensores = {}
3     for ncubo in self.sia_subsistema.ncubos:
4         # print(f"Tensor {i} (forma {ncubo.data.shape}):\n{ncubo.data}\n")
5         tensores[ncubo.indice] = ncubo.data.flatten()
6     return tensores
```

La función *descomponer_en_tensoros* recorre un conjunto de objetos llamados "ncubos" que pertenecen al subsistema *sia_subsistema*. Para cada ncubo, toma su dato almacenado en forma de tensor (una matriz multidimensional), lo aplanado (convierte en un vector unidimensional) y lo guarda en un diccionario. La clave de este diccionario es el índice del ncubo, y el valor es el vector plano resultante. Finalmente, la función retorna este diccionario con todos los tensores aplanados.

```
1 def hamming_distance(self, s1, s2):
2     """Calcula la distancia de Hamming entre dos estados enteros."""
3     return bin(s1 ^ s2).count('1')
```

Esta función calcula la distancia de Hamming entre dos números enteros *s1* y *s2*. La distancia de Hamming es la cantidad de bits diferentes entre dos números cuando se representan en binario.

Para ello, se realiza una operación XOR bit a bit entre ambos números (*s1* ^ *s2*). El resultado es un número donde cada bit que es 1 indica una diferencia entre los bits originales. Luego, se convierte este resultado en una cadena binaria y se cuentan cuántos '1' hay, lo que corresponde al número total de bits diferentes.

```

1 def calcular_tabla_costos(self):
2     if not self.tensores:
3         return {}
4
5     tablas = {}
6     # Variables en el mecanismo (presente)
7     mecanismo = self.sia_subsistema.dims_ncubos
8     num_bits = len(mecanismo)
9     num_states = 2 ** num_bits
10
11     # Estado fuente: solo las variables del mecanismo
12     bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in mecanismo]
13     source_state = self.bits_to_int(bits_fuente)
14
15     for var in self.sia_subsistema.indices_ncubos: # Solo variables en el alcance (futuro)
16         # print(f"Calculando tabla de costos para variable {var}...")
17         tensor = self.tensores[var]
18         fila = np.zeros(num_states)
19         for d in range(1, num_bits + 1):
20             for j in range(num_states):
21                 if self.hamming_distance(source_state, j) == d:
22                     gamma = 2 ** -d
23                     delta = abs(tensor[source_state] - tensor[j])
24                     suma_intermedia = sum(
25                         fila[source_state ^ (1 << k)]
26                         for k in range(num_bits)
27                         if self.hamming_distance(source_state ^ (1 << k), j) == d - 1 and (source_state ^ (1 << k)) < num_states
28                     )
29                     fila[j] = gamma * (delta + suma_intermedia)
30             tablas[var] = fila
31     return tablas

```

La función *calcular_tabla_costos* tiene como objetivo construir una tabla de costos para cada variable del sistema, considerando únicamente aquellas que están dentro del alcance del análisis (futuro del sistema). Esta tabla es fundamental, ya que proporciona una medida del "esfuerzo" requerido para alcanzar distintos estados a partir del estado inicial del sistema.

El proceso se basa en lo siguiente:

- **Mecanismo (Presente):** Se identifican las variables actuales que definen el estado del sistema (variables del mecanismo).
- **Estado inicial:** Se convierte el estado inicial, considerando únicamente las variables del mecanismo, a su representación entera (source_state).
- **Costos por distancia de Hamming:** Para cada posible estado futuro que difiere del estado inicial por d bits (medido con la distancia de Hamming), se calcula un costo incremental. Este depende de dos factores:
 - o La diferencia de valor entre el tensor en el estado inicial y el estado destino (delta).
 - o Una suma intermedia ponderada que incorpora caminos alternativos de transición más cortos, lo cual permite modelar transiciones indirectas.
- **Descuento exponencial:** Se utiliza un factor $\gamma = 2^{-d}$ que penaliza los estados más alejados, haciendo que transiciones con más cambios de bits sean menos influyentes.

Finalmente, se almacena una fila de costos para cada variable, lo cual permite evaluar posteriormente la calidad de distintas biparticiones del sistema.

```

1 def identificar_biparticiones_candidatas(self):
2     if not self.tabla_transiciones:
3         return []
4
5     first_key = next(iter(self.tabla_transiciones))
6     num_states = len(self.tabla_transiciones[first_key])
7     num_bits = (num_states - 1).bit_length()
8     mecanismo = self.sia_subsistema.dims_ncubos
9     bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in mecanismo]
10
11     estado_inicial = self.bits_to_int(bits_fuente)
12
13     candidatas = []
14     #print(f"Identificando biparticiones candidatas para {len(self.sia_subsistema.indices_ncubos)} variables y {num_states} estados...")
15
16     for var in self.sia_subsistema.indices_ncubos:
17         fila = self.tabla_transiciones[var]
18         min_costo = np.min([fila[j] for j in range(num_states) if j != estado_inicial])
19         estados_min = [j for j in range(num_states) if j != estado_inicial and fila[j] == min_costo]
20
21         #print(f"Variable {var} tiene minimo costo {min_costo:.8f} en estados: {' '.join(format(e, f'0{num_bits}b') for e in estados_min)}")
22
23     for estado in estados_min:
24         grupo1 = [var]
25         grupo2 = []
26         estado_inverso = estado ^ ((1 << num_bits) - 1)
27         # Para cada otra variable, ver en qué transición tiene su menor costo
28         #print(f"Evaluando estado {estado} ({format(estado, f'0{num_bits}b')} con minimo costo {min_costo:.8f} en variable {var}")
29         for otra_var in self.sia_subsistema.indices_ncubos:
30             if otra_var == var:
31                 continue
32             fila_otra = self.tabla_transiciones[otra_var]
33             costo_estado = fila_otra[estado]
34             costo_inverso = fila_otra[estado_inverso]
35
36             if costo_estado < costo_inverso:
37                 grupo1.append(otra_var)
38             elif costo_inverso < costo_estado:
39                 grupo2.append(otra_var)
40             else:
41                 grupo1.append(otra_var)
42
43         bits_ini = format(estado_inicial, f'0{num_bits}b')
44         bits_estado = format(estado, f'0{num_bits}b')
45         bits_inverso = format(estado_inverso, f'0{num_bits}b')
46         # print(f"Estado Inicial: {bits_ini}")
47         # print(f"Estado Final : {bits_estado}")
48         # print(f"Estado Inverso: {bits_inverso}")
49
50         # Mecanismo grupo 1: TODAS las variables que NO cambian en la transición estado_inicial -> estado
51         mecanismo_grupo1 = [self.sia_subsistema.dims_ncubos[i] for i in range(num_bits) if bits_ini[i] == bits_estado[i]]
52         # Mecanismo grupo 2: TODAS las variables que NO cambian en la transición estado_inicial -> estado_inverso
53         mecanismo_grupo2 = [self.sia_subsistema.dims_ncubos[i] for i in range(num_bits) if bits_ini[i] == bits_inverso[i]]
54
55         #print(f"Grupo 1: {grupo1} con mecanismos {mecanismo_grupo1}")
56         #print(f"Grupo 2: {grupo2} con mecanismos {mecanismo_grupo2}")
57
58         # Solo considerar biparticiones no triviales
59         if grupo1 and not grupo2:
60             candidatas.append((grupo2, grupo1, mecanismo_grupo2, mecanismo_grupo1))
61         else:
62             candidatas.append((grupo1, grupo2, mecanismo_grupo1, mecanismo_grupo2))
63
64         #print(f"Encontrada bipartición: {grupo1} | {grupo2} con mecanismos {mecanismo_grupo1} | {mecanismo_grupo2}")
65
66     # Elimina duplicados (considerando que {A,B} y {B,A} son iguales)
67     candidatas_unicas = []
68     claves_vistas = set()
69     for c in candidatas:
70         grupoA, grupoB, mecA, mecB = map(frozenset, c)
71         clave = (grupoA, grupoB, mecA, mecB)
72         clave_inv = (grupoB, grupoA, mecB, mecA)
73         if clave not in claves_vistas and clave_inv not in claves_vistas:
74             candidatas_unicas.append(c)
75             claves_vistas.add(clave)
76             claves_vistas.add(clave_inv)
77             #print(f"Encontrada bipartición única: {c}")
78
79     #print(f"Se encontraron {len(candidatas_unicas)} biparticiones candidatas.")
80     return candidatas_unicas
81

```

La función *identificar_biparticiones_candidatas* tiene como objetivo detectar posibles divisiones (biparticiones) de las variables del sistema en dos grupos con base en la tabla de costos previamente calculada. Estas biparticiones

representan formas alternativas de organizar el sistema que podrían reflejar distintas dinámicas o influencias causales.

El proceso se realiza de la siguiente manera:

- **Estado inicial y mecanismo:** Se parte del estado actual del sistema (estado fuente) codificado en binario, y se identifican las variables del mecanismo (presente).
- **Selección de estados de mínimo costo:** Para cada variable en el alcance (futuro), se identifica el estado al que se puede llegar desde el estado inicial con el menor costo, según la tabla de transiciones. También se calcula su estado inverso (complemento bit a bit).
- **Agrupamiento de variables:** Se agrupan las variables según cuál de los dos estados (normal o inverso) representa una transición de menor costo. Esto da lugar a dos grupos: uno que comparte preferencia por el estado original, y otro por el estado inverso.
- **Determinación de mecanismos asociados:** Para cada grupo, se determina el subconjunto del mecanismo que permanece sin cambio entre el estado inicial y el estado de destino (o su inverso).
- Al final del proceso, la función organiza las biparticiones generadas para mantener un formato uniforme. En particular, asegura que el primer grupo contenga la menor cantidad de variables y el segundo grupo el resto, lo cual permite estandarizar la representación de cada bipartición. Esta organización también evita duplicidades lógicas, ya que biparticiones equivalentes como (A, B) y (B, A) se representan de manera consistente y única.

Este proceso da como resultado un conjunto de biparticiones no triviales y únicas, que podrán ser posteriormente evaluadas para determinar cuál describe mejor la organización funcional del sistema.


```

1 def identificar_biparticiones_candidatas_extra(self, candidatos):
2     if not self.tabla_transiciones:
3         return candidatos
4
5     first_key = next(iter(self.tabla_transiciones))
6     num_states = len(self.tabla_transiciones[first_key])
7     num_bits = (num_states - 1).bit_length()
8     mecanismo = self.sia_subsistema.dims_ncubos
9     bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in mecanismo]
10
11     estado_inicial = self.bits_to_int(bits_fuente)
12
13     num_bits = len(mecanismo)
14     estado_complementario = estado_inicial ^ ((1 << num_bits) - 1)
15
16     # Buscar variables con costo mínimo en el estado complementario
17     costos_complementarios = [self.tabla_transiciones[var][estado_complementario] for var in self.sia_subsistema.indices_ncubos]
18
19     max_costo = max(costos_complementarios)
20     umbral = ANALYSIS_PERCENTAGE * max_costo
21     vars_min = [var for var, costo in zip(self.sia_subsistema.indices_ncubos, costos_complementarios) if costo <= umbral]
22     # print(f"Variables con costo mínimo (min_costo: {8f}) en estado complementario {estado_complementario} ({format(estado_complementario, f'0{num_bits}b')}): {vars_min}")
23
24     # print(vars_min)
25
26     for var in vars_min:
27         grupo1 = [var]
28         grupo2 = [v for v in self.sia_subsistema.indices_ncubos if v != var]
29
30         # print(f"Evaluando variable {var} con costo mínimo (min_costo: {8f}) en estado complementario {estado_complementario} ({format(estado_complementario, f'0{num_bits}b')}")
31
32         bits_ini = format(estado_inicial, f'0{num_bits}b')
33         bits_comp = format(estado_complementario, f'0{num_bits}b')
34
35         # Mecanismo grupo 1: TODAS las variables que NO cambian en la transición estado_inicial -> estado_inverso
36         mecanismo_grupo1 = [self.sia_subsistema.dims_ncubos[i] for i in range(num_bits) if bits_ini[i] == bits_comp[i]]
37         # Mecanismo grupo 2: TODAS las variables que NO cambian en la transición estado_inicial -> estado_inicial
38         mecanismo_grupo2 = [self.sia_subsistema.dims_ncubos[i] for i in range(num_bits)]
39
40         # Solo considerar biparticiones no triviales
41         if grupo1 and not grupo2:
42             candidatos.append((grupo2, grupo1, mecanismo_grupo2, mecanismo_grupo1))
43         else:
44             candidatos.append((grupo1, grupo2, mecanismo_grupo1, mecanismo_grupo2))
45
46         # print(f"Bipartición extra agregada {candidatos[-1]}")
47         # print(f"Encontrada bipartición extra: {grupo1} | {grupo2} con mecanismos {mecanismo_grupo1} | {mecanismo_grupo2}")
48
49     # Elimina duplicados (considerando que (A,B) y (B,A) son iguales)
50
51     # print(f"Se encontraron {len(candidatos)} biparticiones candidatas (incluyendo extra).")
52     return candidatos

```

La función *identificar_biparticiones_candidatas_extra* complementa el conjunto de biparticiones previamente hallado, explorando transiciones adicionales que puedan resultar informativas, especialmente aquellas que parten del estado inicial hacia su complemento binario (estado complementario). Esta estrategia busca captar relaciones causales menos evidentes o indirectas.

El procedimiento es el siguiente:

- **Estado complementario:** Se calcula el estado binario inverso (complementario) al estado inicial, invirtiendo cada uno de sus bits.
- **Evaluación de costos en el estado complementario:** Para cada variable del sistema, se obtiene el costo de transición hacia este estado complementario. Luego se seleccionan aquellas variables cuyo costo se encuentra por debajo de un umbral porcentual (*ANALYSIS_PERCENTAGE*) del costo máximo observado. Estas variables se consideran relevantes en esta transición alternativa.
- **Formación de grupos:** Cada una de estas variables con bajo costo en el estado complementario se considera como un grupo individual (grupo 1), y las demás variables forman el grupo 2. Esto permite construir biparticiones centradas en la influencia específica de variables individuales bajo condiciones distintas.
- **Determinación de mecanismos asociados:** Para cada grupo se determina el subconjunto del mecanismo que permanece sin cambio entre el estado inicial y

el complementario (para el grupo 1) y entre el estado inicial y sí mismo (grupo 2).

- **Organización de bipartición:** Las biparticiones son organizadas para favorecer los grupos de pocas variables en el grupo 1 y el resto en el grupo 2, para que todo tenga una estructura uniforme que posteriormente se usará para filtrar los candidatos.

Esta función permite enriquecer el conjunto de biparticiones exploradas, ampliando el análisis hacia configuraciones que podrían no haberse detectado mediante transiciones directas desde el estado inicial.

```
1 def filtrar_candidatos_por_tamano(self, candidatos):
2     if not candidatos:
3         return []
4
5     min_size = min(len(c[0]) + len(c[2]) for c in candidatos)
6     max_size = max(len(c[0]) + len(c[2]) for c in candidatos)
7
8     if MAX_VAR == 0:
9         umbral = min_size + math.floor((max_size - min_size) / 2)
10    elif MAX_VAR > min_size and MAX_VAR < max_size:
11        umbral = MAX_VAR
12    elif MAX_VAR == -1:
13        umbral = min_size + 2
14    else:
15        umbral = max_size
16
17    filtrados = [c for c in candidatos if (len(c[0]) + len(c[2])) <= umbral]
18    #print(f'Filtrando candidatos: tamaño mínimo {min_size}, máximo {max_size}, umbral {umbral}. Quedan {len(filtrados)} de {len(candidatos)}.')
19    return filtrados
```

La función ***filtrar_candidatos_por_tamano*** se encarga de reducir el conjunto de biparticiones candidatas generadas previamente, aplicando un criterio basado en el tamaño combinado de los grupos de cada bipartición. El objetivo de este filtrado es limitar la complejidad del análisis posterior, evitando evaluar biparticiones excesivamente grandes que podrían ser costosas o poco informativas.

Para cada bipartición candidata se calcula la suma de tamaños del primer grupo de variables ($c[0]$) y del subconjunto de mecanismo asociado ($c[2]$). Con estos valores se determina el tamaño mínimo y máximo entre todas las biparticiones. A partir de esa información, se establece un umbral que define qué biparticiones serán consideradas válidas para el análisis. Este umbral se calcula de acuerdo con el valor de la constante MAX_VAR :

- Si $MAX_VAR == 0$, se utiliza un punto intermedio entre el tamaño mínimo y el máximo.
- Si MAX_VAR está dentro del rango entre el mínimo y el máximo, se toma directamente como umbral.
- Si $MAX_VAR == -1$, se permite un poco más que el mínimo (mínimo + 2).

En cualquier otro caso, se acepta el tamaño máximo posible. Una vez definido el umbral, se filtran todas las biparticiones cuya suma de tamaños de grupo y mecanismo no lo exceda. El resultado es un subconjunto más pequeño y controlado de biparticiones candidatas, optimizado para continuar con el análisis sin comprometer la eficiencia computacional ni la diversidad estructural del conjunto.

```
1  def evaluar_biparticiones(self, candidatos):
2      mejor = None
3      mejor_costo = float('inf')
4      mejor_dist = None
5      memoria_particiones = {}
6
7      for A, B, a, b in candidatos:
8          clave = (frozenset(A), frozenset(a))
9          if clave in memoria_particiones:
10             # print(f"Usando memoria para clave {clave}")
11             costo, dist = memoria_particiones[clave]
12          else:
13             costo, dist = self.evaluar_coste_biparticion(A, a)
14             memoria_particiones[clave] = (costo, dist)
15
16             if costo < mejor_costo:
17                 mejor = (A, B, a, b)
18                 mejor_costo = costo
19                 mejor_dist = dist
20
21      return mejor, mejor_dist, mejor_costo
```

La función *evaluar_biparticiones* se encarga de recorrer el conjunto de biparticiones candidatas y determinar cuál de ellas representa la mejor separación del sistema, con base en un criterio de costo. Este costo refleja la discrepancia entre la distribución marginal generada por la bipartición y la distribución marginal original del sistema, y se calcula utilizando la función *evaluar_coste_biparticion*. Durante el proceso, se utiliza una estructura de memoria (*memoria_particiones*) para evitar cálculos repetidos. Esto es posible porque una bipartición está determinada únicamente por su combinación de subconjuntos de variables futuras (A) y presentes (a). Si esa combinación ya ha sido evaluada previamente, se reutiliza el resultado almacenado; de lo contrario, se realiza el cálculo del costo y la distribución marginal asociada, y se guarda para usos futuros.

La función mantiene un seguimiento de la mejor bipartición encontrada hasta el momento, comparando el costo actual con el menor costo registrado. Si se encuentra una bipartición con menor costo, esta se almacena como la nueva mejor opción. Al finalizar, se retorna la mejor bipartición encontrada junto con su distribución marginal y su costo correspondiente. Este proceso es esencial para identificar la estructura más informativa y eficiente dentro del espacio de biparticiones generadas, y su diseño evita redundancias computacionales gracias al uso inteligente de memoria intermedia.

```
1 def evaluar_coste_biparticion(self, futuro_A, presente_a):
2     particion = self.sia_subsistema.bipartir(np.array(futuro_A), np.array(presente_a))
3     dist = particion.distribucion_marginal()
4
5     costo = emd_efecto(dist, self.sia_dists_marginales)
6
7     return costo, dist
```

La función *evaluar_coste_biparticion* permite cuantificar qué tan informativa o efectiva es una determinada bipartición del sistema, calculando el costo asociado a la separación de variables en función de su capacidad para representar correctamente la dinámica del sistema.

El procedimiento es el siguiente:

- **Construcción de la bipartición:** A partir de los conjuntos futuro_A (variables futuras del grupo A) y presente_a (variables presentes del grupo a), se construye una bipartición del sistema mediante el método *bipartir* del subsistema SIA. Esta operación genera una representación dividida del sistema en dos subconjuntos de variables.
- **Distribución marginal resultante:** Se calcula la distribución marginal asociada a la bipartición, lo cual permite observar cómo se redistribuyen las probabilidades bajo esta separación del sistema.
- **Evaluación del costo:** Se utiliza la métrica EMD (Earth Mover's Distance) para comparar la distribución marginal resultante con la distribución marginal original del sistema (self.sia_dists_marginales). Esta métrica mide la "distancia" entre ambas distribuciones, interpretándose como un costo de pérdida de información o distorsión al aplicar la bipartición.

- **Resultado:** Se retorna tanto el costo calculado como la distribución marginal correspondiente, lo cual permite evaluar múltiples biparticiones y seleccionar las más prometedoras en términos de bajo costo informativo.

Esta función es clave para determinar la calidad de una bipartición, permitiendo priorizar aquellas configuraciones que mejor preservan las relaciones estadísticas del sistema original.

```

1  def guardar_tabla_costos_excel(self):
2      if not self.tabla_transiciones:
3          return
4
5      output_dir = self.sia_gestor.output_dir
6      output_dir.mkdir(parents=True, exist_ok=True)
7      file_path = output_dir / "costos_desde_estado_inicial.xlsx"
8
9      first_key = next(iter(self.tabla_transiciones))
10     num_states = len(self.tabla_transiciones[first_key])
11     num_bits = (num_states - 1).bit_length()
12     estado_inicial = self.bits_to_int(self.sia_subsistema.estado_inicial)
13
14     columnas = []
15     for k in self.tabla_transiciones:
16         if hasattr(self, 'sia_condicion_nodos_nombres') and k < len(self.sia_condicion_nodos_nombres):
17             columnas.append(self.sia_condicion_nodos_nombres[k])
18         elif k < len(ABECEDARY):
19             columnas.append(ABECEDARY[k])
20         else:
21             columnas.append(str(k))
22
23     datos = []
24     filas = []
25     for j in range(num_states):
26         # if j == estado_inicial:
27             # continue
28         fila = [self.tabla_transiciones[k][j] for k in self.tabla_transiciones]
29         datos.append(fila)
30         filas.append(f"{format(estado_inicial, f'0{num_bits}b')} > {format(j, f'0{num_bits}b')}")
31
32     df = pd.DataFrame(datos, columns=columnas, index=filas)
33
34     with pd.ExcelWriter(file_path, engine='xlsxwriter') as writer:
35         df.to_excel(writer, sheet_name='CostosTransiciones')
36         ws = writer.sheets['CostosTransiciones']
37         ws.conditional_format(1, 1, len(df), len(df.columns), {
38             'type': '2_color_scale', 'min_color': "#FFFFFF", 'max_color': "#F8696B"
39         })
40
41     print(f"Tabla de costos guardada en {file_path}")
42

```

Esta función tiene como propósito generar una tabla de costos en formato Excel, con el fin de verificar la veracidad de los datos o validar los resultados obtenidos a partir de las transiciones de estado calculadas. La tabla representa los

costos de transición desde un estado inicial hacia otros posibles estados, organizados de manera comprensible.

3.2. Clase `geometric_mp.py` – Paralelo

```
1 def calcular_tabla_costos(self):
2     if not self.tensores:
3         return {}
4
5     mecanismo = self.sia_subsistema.dims_ncubos
6     num_bits = len(mecanismo)
7     num_states = 2 ** num_bits
8     bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in mecanismo]
9     source_state = int(''.join(str(b) for b in bits_fuente), 2)
10
11     tasks_args_list = []
12     for var_key in self.sia_subsistema.indices_ncubos:
13         tensor_val = self.tensores[var_key]
14         tasks_args_list.append((var_key, tensor_val.copy(), num_states, num_bits, source_state, mecanismo))
15
16     with multiprocessing.Pool() as pool:
17         resultados = pool.map(calcular_fila, tasks_args_list)
18     tablas = {var: fila for var, fila in resultados}
19     return tablas
```

La función *calcular_tabla_costos* tiene como propósito construir una tabla de costos de transición desde el estado inicial hacia todos los posibles estados del sistema, para cada variable en análisis. Esta versión utiliza el módulo `multiprocessing` de Python para paralelizar el proceso y mejorar el rendimiento.

- **Identificación del mecanismo:** Se obtienen las variables del mecanismo (dimensiones presentes) y se calcula la cantidad total de estados posibles (2^n), así como el estado inicial (`source_state`) en formato entero.
- **Preparación de tareas:** Para cada variable del alcance (futuro), se genera una tupla de argumentos que incluye su tensor correspondiente y los parámetros necesarios para el cálculo de costos.
- Se crea un pool de procesos mediante *`multiprocessing.Pool`* y se distribuye la ejecución de las tareas utilizando `pool.map`, que aplica la función *calcular_fila* a cada conjunto de argumentos.

```

1 def calcular_fila(args):
2     var, tensor, num_states, num_bits, source_state, mecanismo = args
3     fila = np.zeros(num_states)
4     for d in range(1, num_bits + 1):
5         for j in range(num_states):
6             if bin(source_state ^ j).count('1') == d:
7                 gamma = 2 ** -d
8                 delta = abs(tensor[source_state] - tensor[j])
9                 suma_intermedia = sum(
10                     fila[source_state ^ (1 << k)]
11                     for k in range(num_bits)
12                     if bin((source_state ^ (1 << k)) ^ j).count('1') == d - 1 and (source_state ^ (1 << k)) < num_states
13                 )
14                 fila[j] = gamma * (delta + suma_intermedia)
15     return (var, fila)

```

Esta función auxiliar calcula, para una sola variable, los costos de transición hacia todos los estados posibles, teniendo en cuenta la distancia de Hamming, un factor de penalización exponencial y rutas intermedias de menor costo.

Esta estrategia de paralelización permite reducir significativamente el tiempo de cómputo, especialmente en sistemas con un número elevado de variables o estados posibles, manteniendo la precisión del cálculo y la estructura del algoritmo original.


```

1  def identificar_biparticiones_candidatas(self):
2      if not self.tabla_transiciones:
3          return []
4
5      mecanismo = self.sia_subsistema.dims_ncubos
6      indices_ncubos = self.sia_subsistema.indices_ncubos
7      num_bits = len(mecanismo)
8      num_states = 2 ** num_bits
9      bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in mecanismo]
10     estado_inicial = int(''.join(str(b) for b in bits_fuente), 2)
11
12     args_list = [
13         (var, self.tabla_transiciones, num_bits, num_states, estado_inicial, indices_ncubos, mecanismo)
14         for var in indices_ncubos
15     ]
16
17     with multiprocessing.Pool() as pool:
18         resultados = pool.map(biparticion_candidata_worker, args_list)
19
20     candidatas = []
21     for sublist in resultados:
22         candidatas.extend(sublist)
23
24     # Elimina duplicados (considerando que (A,B) y (B,A) son iguales)
25     candidatas_unicas = []
26     claves_vistas = set()
27     for c in candidatas:
28         grupoA, grupoB, mecA, mecB = map(frozenset, c)
29         clave = (grupoA, grupoB, mecA, mecB)
30         clave_inv = (grupoB, grupoA, mecB, mecA)
31         if clave not in claves_vistas and clave_inv not in claves_vistas:
32             candidatas_unicas.append(c)
33             claves_vistas.add(clave)
34             claves_vistas.add(clave_inv)
35     # print(f"Se encontraron {len(candidatas_unicas)} biparticiones candidatas.")
36     return candidatas_unicas
37

```

La función *identificar_biparticiones_candidatas* está diseñada para detectar todas las biparticiones potencialmente relevantes dentro del sistema, a partir de una tabla de costos previamente calculada. Su objetivo es dividir las variables del sistema en dos grupos distintos que reflejen diferencias significativas en el comportamiento observado desde el estado inicial. Esta versión está optimizada mediante procesamiento paralelo utilizando el módulo multiprocessing, lo cual permite mejorar significativamente el rendimiento en sistemas con muchas variables.

El procedimiento inicia con una validación: si no existe una tabla de transiciones, no hay base para generar biparticiones y se retorna una lista vacía. A continuación, se extrae la configuración actual del subsistema, incluyendo las variables del mecanismo (mecanismo) y del alcance (indices_ncubos). También se determina el número de bits necesarios para representar los estados (num_bits) y se calcula la cantidad total de estados posibles ($2^{\text{num_bits}}$). Luego, a partir del estado inicial del sistema, se construye su representación binaria como un número

entero (estado_inicial) ***Nota: Hasta acá es el mismo proceso que la versión secuencial.***

Con esta información, se prepara una lista de argumentos que serán distribuidos entre los procesos del pool dependiendo del número de tensores existentes. Cada entrada representa una tarea para evaluar una variable individual y generar biparticiones a partir de los estados con menor costo asociados a esa variable/tensor. Estas tareas son ejecutadas en paralelo mediante pool.map, utilizando una función externa (***_biparticion_candidata_worker***) que se encarga de construir, por cada variable, las posibles biparticiones entre grupos de variables que prefieren un estado sobre su complemento. Si hay más tensores que núcleos en el sistema, estos quedan en “cola”. El resultado de esta etapa es una lista de listas, que luego se consolida en una única lista de candidatas.

Finalmente, se aplica el mismo proceso de filtrado descrito previamente en la función de ***filtrar_candidatos_por_tamano***. Este proceso permite eliminar duplicados al considerar que biparticiones como (A, B) y (B, A) representan la misma separación lógica. Para ello, se emplean conjuntos inmutables (frozenset) que permiten comparar las combinaciones sin importar el orden, y se lleva un registro de las claves ya vistas mediante un conjunto (set). De este modo, sólo se conservan las biparticiones únicas en el resultado final, lo que garantiza un conjunto depurado de candidatos que podrán ser evaluados posteriormente según su costo. Esta etapa resulta esencial para la selección de estructuras causales o funcionales dentro del sistema.

Se conservan exactamente las mismas funciones ***identificar_biparticiones_candidatas_extra*** y ***filtrar_candidatos_por_tamano*** que se utilizaron en la implementación secuencial.

```

1  def evaluar_biparticiones(self, candidatos):
2      mejor = None
3      mejor_costo = float('inf')
4      mejor_dist = None
5
6      # Prepara los argumentos para cada proceso
7      args_list = [
8          (A, B, a, b, self.sia_subsistema, self.sia_dists_marginales)
9          for (A, B, a, b) in candidatos
10     ]
11
12     with multiprocessing.Pool() as pool:
13         resultados = pool.map(_evaluar_biparticion_worker, args_list)
14
15     for costo, dist, datos in resultados:
16         if costo < mejor_costo:
17             mejor = datos
18             mejor_costo = costo
19             mejor_dist = dist
20
21     return mejor, mejor_dist, mejor_costo
22

```

La función *evaluar_biparticiones* tiene como objetivo seleccionar la mejor bipartición entre un conjunto de candidatas, basándose en el costo de cada una. Para ello, se construye una lista de argumentos que incluye cada bipartición candidata junto con la información necesaria del subsistema y su distribución marginal de referencia. Esta lista se pasa a un pool de procesos creado con `multiprocessing.Pool`, lo que permite evaluar todas las biparticiones en paralelo según la cantidad de núcleos mediante la función auxiliar *_evaluar_biparticion_worker*.



```
1 def _evaluar_biparticion_worker(args):
2     A, B, a, b, sia_subsistema, sia_dists_marginales = args
3     particion = sia_subsistema.bipartir(np.array(A), np.array(a))
4     dist = particion.distribucion_marginal()
5     costo = emd_efecto(dist, sia_dists_marginales)
6     return costo, dist, (A, B, a, b)
```

Cada evaluación calcula una medida de costo (usualmente a través de la distancia entre distribuciones marginales), y una vez recogidos todos los resultados, la función compara los costos obtenidos para identificar cuál bipartición presenta el valor más bajo.

Finalmente, se retorna la mejor bipartición junto con su distribución marginal y su costo asociado. Esta implementación mejora notablemente la eficiencia frente a una evaluación secuencial, especialmente cuando el número de candidatos es alto.

3.3. Clase `geometric_cuda.py` – Paralelo

```
1 def calcular_tabla_costos_cuda(self):
2     tablas = {}
3     num_bits = len(self.sia_subsistema.dims_ncubos)
4     num_states = 1 << num_bits
5
6     bits_fuente = [self.sia_subsistema.estado_inicial[i] for i in self.sia_subsistema.dims_ncubos]
7     source_state = int("".join(str(b) for b in bits_fuente), 2)
8
9     threads_per_block = 256
10
11     # Precalcular conjuntos de estados por distancia
12     distancias_j = [[] for _ in range(num_bits + 1)]
13     for j in range(num_states):
14         d = bin(source_state ^ j).count('1')
15         distancias_j[d].append(j)
16
17     for var, tensor_flat in self.tensores.items():
18         fila = np.zeros(num_states, dtype=np.float32)
19
20         d_tensor = cuda.to_device(tensor_flat.astype(np.float32))
21         d_fila = cuda.to_device(fila)
22
23         for d in range(1, num_bits + 1):
24             js = np.array(distancias_j[d], dtype=np.int32)
25             if js.size == 0:
26                 continue
27
28             d_js = cuda.to_device(js)
29             d_fila_prev = d_fila # ya tiene lo anterior
30
31             blocks_per_grid = (js.size + threads_per_block - 1) // threads_per_block
32
33             calcular_costos_por_distancia_progresiva[blocks_per_grid, threads_per_block](
34                 d_tensor, source_state, d_fila, d_fila_prev, d_js, num_bits
35             )
36             cuda.synchronize()
37
38             del d_js
39
40             d_fila.copy_to_host(fila)
41             tablas[var] = fila
42
43             del d_tensor
44             del d_fila
45
46         cuda.current_context().memory_manager.deallocations.clear()
47         gc.collect()
48
49     return tablas
```

La función *calcular_tabla_costos_cuda* implementa una versión altamente optimizada del cálculo de la tabla de costos, utilizando procesamiento en GPU mediante la biblioteca *numba.cuda*. Esta versión está diseñada para aprovechar la arquitectura paralela masiva de las GPUs, permitiendo evaluar múltiples transiciones entre estados de manera simultánea y extremadamente eficiente. El proceso inicia obteniendo el número total de bits y estados posibles según las

variables del mecanismo del sistema, además de codificar el estado inicial como un entero. A continuación, se precalcula un conjunto de listas que agrupan todos los estados posibles según su distancia de Hamming con respecto al estado inicial. Esto organiza el trabajo de forma que se puedan lanzar kernels CUDA por niveles de distancia. Finalmente se sincronizan los cálculos para asegurar que el uso de costos intermedios quede listo para el siguiente grupo.

Para cada variable en el sistema, se prepara un vector de costos inicializado en ceros (fila), que será calculado en la GPU. El tensor correspondiente a la variable y el vector de costos son transferidos a memoria de GPU. Luego, para cada nivel de distancia d , se filtran los estados a evaluar (js) y se lanza un kernel CUDA (*calcular_costos_por_distancia_progresiva*) que calcula los costos correspondientes de forma completamente paralela en bloques e hilos. Una vez procesadas todas las distancias, los resultados se copian de vuelta al host y se almacenan en la tabla final.

```
1 @cuda.jit
2 def calcular_costos_por_distancia_progresiva(tensor, source_state, fila, fila_prev, js, num_bits):
3     idx = cuda.grid(1)
4     if idx >= js.shape[0]:
5         return
6
7     j = js[idx]
8
9     # Distancia de Hamming entre j y source_state (fija para esta capa)
10    xor = source_state ^ j
11    dist = 0
12    while xor:
13        dist += xor & 1
14        xor >>= 1
15
16    gamma = 2.0 ** -dist
17    delta = abs(tensor[source_state] - tensor[j])
18
19    suma_intermedia = 0.0
20    for k in range(num_bits):
21        vecino = source_state ^ (1 << k)
22        if vecino >= fila_prev.shape[0]:
23            continue
24
25        xor2 = vecino ^ j
26        dist2 = 0
27        while xor2:
28            dist2 += xor2 & 1
29            xor2 >>= 1
30
31        if dist2 == dist - 1:
32            suma_intermedia += fila_prev[vecino]
33
34    fila[j] = gamma * (delta + suma_intermedia)
```

Además, el cálculo paralelo en la GPU se organiza en una grilla de bloques, donde cada bloque contiene múltiples hilos. Cada hilo se encarga de calcular el costo para un estado específico dentro del conjunto de estados a evaluar (js) para una distancia d determinada. El número de bloques se ajusta dinámicamente según la cantidad de estados a procesar y el tamaño de los hilos por bloque definidos (*threads_per_block*), lo que permite aprovechar al máximo la capacidad de cómputo de la GPU.

Finalmente, se libera la memoria de GPU y se fuerza la recolección de basura para evitar pérdidas de memoria. Esta implementación es ideal para sistemas con una gran cantidad de variables o estados, donde las versiones secuencial o multiproceso resultan insuficientes en rendimiento.

Seguidamente se hace uso nuevamente de las funciones *identificar_biparticiones_candidatas*, *identificar_biparticiones_candidatas_extra* y *filtrar_candidatos_por_tamano* de la implementación en secuencial.

La función *evaluar_biparticiones* (la misma que en la versión con **multiprocessing**) se encarga de determinar cuál bipartición candidata, entre todas las posibles generadas previamente, representa la mejor separación del sistema en términos de costo. En esta versión, el proceso está paralelizado utilizando multiprocessing, lo que permite evaluar múltiples biparticiones de manera simultánea, distribuyendo el trabajo entre distintos núcleos de la CPU. Para ello, se construye una lista de argumentos donde cada elemento representa una bipartición a evaluar, incluyendo los grupos de variables futuras y presentes, el subsistema SIA y la distribución marginal de referencia. Esta lista se pasa a un pool de procesos que ejecutan, en paralelo, la función auxiliar *evaluar_biparticion_worker*. Cada trabajador calcula el costo de su bipartición comparando la distribución marginal resultante con la distribución original, generalmente utilizando una métrica como la Earth Mover's Distance (EMD). Luego de obtener todos los resultados, la función selecciona la bipartición con el menor costo, y retorna sus datos junto con la distribución correspondiente. Esta implementación mejora notablemente la eficiencia en comparación con una evaluación secuencial, especialmente cuando el número de biparticiones candidatas es elevado, permitiendo una exploración más rápida y efectiva del espacio de soluciones.

4. Pruebas y resultados

Para evaluar el impacto de las estrategias de paralelización implementadas, se realizaron pruebas comparativas utilizando tres versiones del sistema: la versión secuencial original (GEOMETRIC), una versión optimizada con paralelización en CPU mediante multiprocessing (GEOMETRIC + Multiprocessing), y una versión que adicionalmente integra paralelización con GPU mediante CUDA (GEOMETRIC + Multiprocessing + CUDA). Los resultados se analizan en términos de dos métricas clave: discrepancia de pérdida y speedup promedio. Adicionalmente se analiza el uso de recursos de la cpu con cada una de las estrategias.

4.1. Discrepancia de pérdida

PROMEDIO DISCREPANCIA DE PERDIDA									
	GEOMETRIC			GEOMETRIC + Multiprocessing			GEOMETRIC + Multiprocessing + CUDA		
Tamaño	MIN	PROM	MAX	MIN	PROM	MAX	MIN	PROM	MAX
3	0.00%	7.65%	25.00%	0.00%	7.65%	25.00%			
5	0.00%	1.80%	50.00%	0.00%	1.80%	50.00%			
10	0.00%	0.04%	2.08%	0.00%	0.04%	2.08%			
15	0.00%	1.00%	3.00%	0.00%	1.00%	3.00%	0.00%	1.00%	3.00%
20	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
21									
Promedio	0.00%	2.10%	16.02%	0.00%	2.10%	16.02%	0.00%	0.50%	1.50%

Esta métrica mide qué tanto difieren los resultados de cada versión respecto a los obtenidos por las referencias estándar del sistema: Phi o QNodes (dependiendo el número de variables, $\Phi \leq 15N$, $Q > 20$). Es decir, se compara la calidad de las particiones encontradas en cada versión frente a las obtenidas por estas implementaciones de referencia. Como se observa en la primera tabla, las tres estrategias presentan una discrepancia igual, lo que indica que las tres emplean el mismo proceso pero con ciertas optimizaciones que refleja la siguiente tabla. Aunque estos resultados son generalmente cercanos, existen desviaciones relevantes en algunos tamaños del sistema. Esta inconsistencia es mayormente vista en tamaños de sistema pequeños. En 21 nodos solo se ejecutó las 3 versiones del proyecto porque Q tarda demasiado y Phi (Directamente no ejecuta), por eso no hay comparación de pérdida, pero podríamos deducir que seguiría la misma tendencia con una discrepancia mínima.

4.2. Speedup promedio

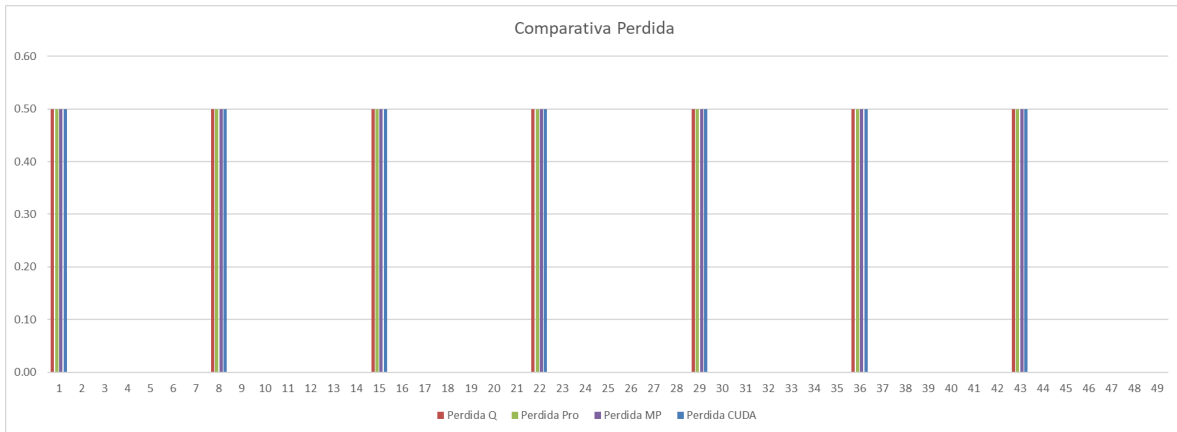
PROMEDIO SPEEDUP									
	GEOMETRIC			GEOMETRIC + Multiprocessing			GEOMETRIC + Multiprocessing + CUDA		
Tamaño	MIN	PROM	MAX	MIN	PROM	MAX	MIN	PROM	MAX
3	0.01	0.49	1.00	-15.94	-11.46	-10.66			
5	-0.01	0.33	1.00	-1475.00	-170.13	-9.47			
10	0.12	298.79	1408.00	-1371.00	-495.70	-12.30			
15	-7.09	120.13	1940.00	-1161.00	-152.60	-1.88	-438.00	-57.75	1.14
20	-1.86	4.76	21.04	-112.80	-19.53	4.67	-44.45	-0.25	18.72
21	0.00	0.00	0.00	-85.82	-9.25	6.01	-36.82	7.85	46.86
Promedio	-1.47	70.75	561.84	-703.59	-143.11	-3.94	-173.09	-16.72	22.24

En cuanto al speedup, esta métrica evalúa el tiempo de ejecución comparado con versiones base. Para la versión secuencial GEOMETRIC, el speedup se calcula comparado con Phi o QNodes, mostrando mejoras notables en algunos casos (por ejemplo, 561.84x en tamaño 10), aunque con cierta variabilidad sobre todo en pruebas que involucren menos variables. En cambio, el speedup de las versiones GEOMETRIC + Multiprocessing y GEOMETRIC + Multiprocessing + CUDA se calcula en comparación directa con GEOMETRIC, para medir el impacto real de las estrategias de paralelización. En este sentido, la versión Multiprocessing muestra resultados inconsistentes, con un promedio negativo de -142.65x, indicando que el costo de la paralelización en CPU no siempre se justifica, especialmente en sistemas de menor tamaño. Por el contrario, la versión CUDA ofrece un comportamiento más equilibrado, con un promedio de -16.99x, y valores máximos positivos en casos de mayor tamaño (hasta 13.67x de mejora), incluso notamos que para tamaños más grandes, el promedio va aumentando, como se nota en los 21 nodos que el promedio es de 7.03x, lo que confirma que su ventaja se acentúa en estructuras más complejas.

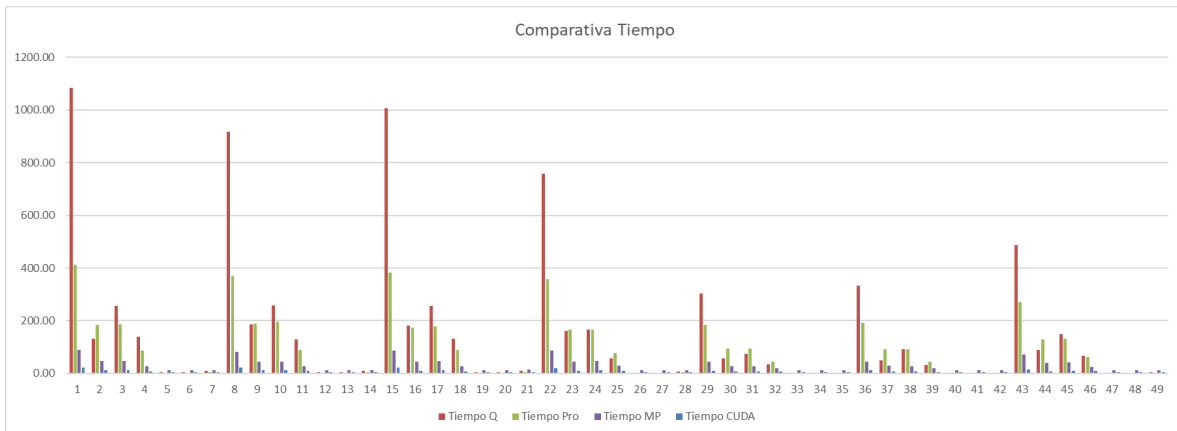
4.3. Comparativa Estrategias (20N)

En esta sección analizaremos el impacto de las 3 estrategias en las pruebas realizadas a los subsistemas de 20 Nodos. Donde podemos comparar estas mismas con una preexistente como lo es QNodes y ver que tan eficiente es nuestra solución.

Primeramente, analizamos la calidad de los resultados:



Como podemos apreciar, prácticamente tuvimos una diferencia del 0% con respecto a QNodes lo que nos asegura que las particiones encontradas son de altísima calidad. Con esta comparación hecha, podemos ahora comparar el tiempo que le tomó a cada algoritmo llegar a esta solución, lo que quedaria de la siguiente manera:



Como se aprecia en esta ocasión, QNodes en la gran mayoría de casos es el que más tiempo toma (a excepción de algunas casos concretos) lo que nos indica el nivel de eficiencia de nuestras estrategias las cuales apreciamos mejor en la siguiente tabla:

0.00%	0.00%	0.00%	4.76	-19.53	0.72	-0.25	10.69
0.00%	0.00%	0.00%	-1.86	-112.80	-13.12	-44.45	-5.20
0.00%	0.00%	0.00%	21.04	4.67	12.25	18.72	49.10
% Perdida Pro	% Perdida MP	% Perdida CUDA	SpeedUp Pro (X)	SpeedUp MP (X)	MP vs Q (X)	SpeedUp CUDA (X)	CUDA vs Q (X)

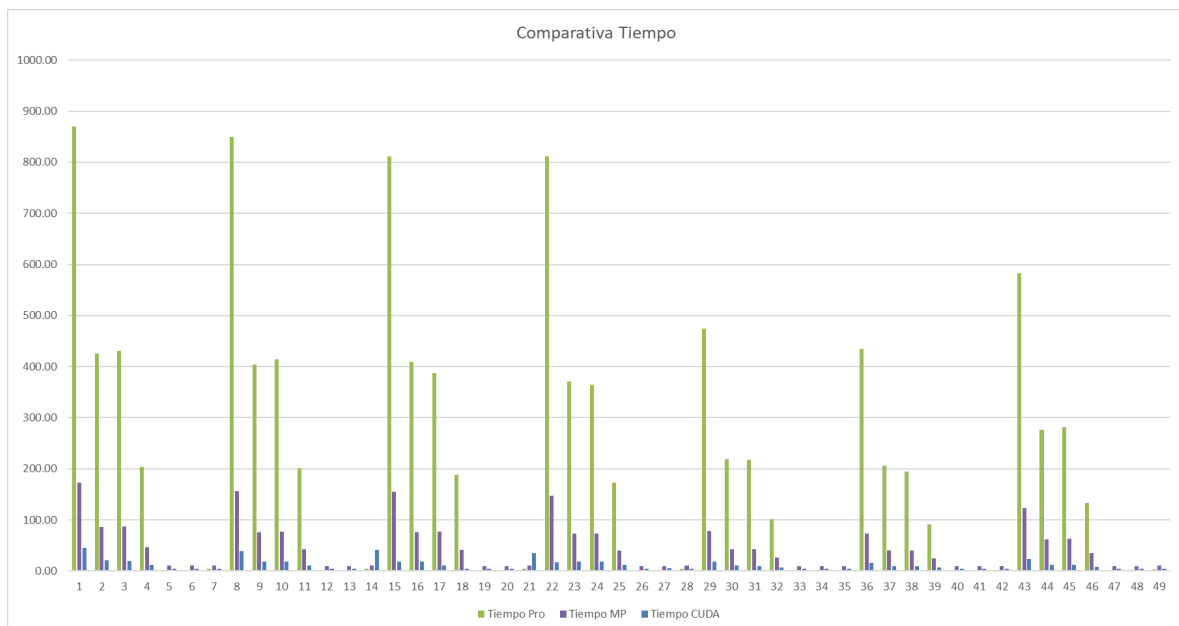
En esta tabla podemos apreciar 3 filas que corresponden al PROMEDIO (amarillo), VERDE (Minimo) y ROJO (Maximo) en distintas metricas. Podemos apreciar nuevamente el 0% de pérdida con respecto a QNodes y más importante que tanto aceleramos con respecto a Geometric Secuencial y QNodes. Podemos apreciar como:

1. En secuencial consigue un speedup Máximo de 21x con respecto a QNodes
2. En paralelo con Multiprocessing consigue un máximo de 12x con respecto a QNodes pero un 4.6x con respecto a secuencial.
3. Finalmente en paralelo con CUDA obtenemos un máximo de 49x con respecto a QNodes y un asombroso 18.7x con respecto a secuencial.

Comparacion Tiempos 20N			
Qnodes (s)	Geometric (s)	Geometric MP (s)	Geometric MP CUDA (s)
1083.60	413.19	88.44	22.07
SpeedUp (x)	2.62	12.25	49.10

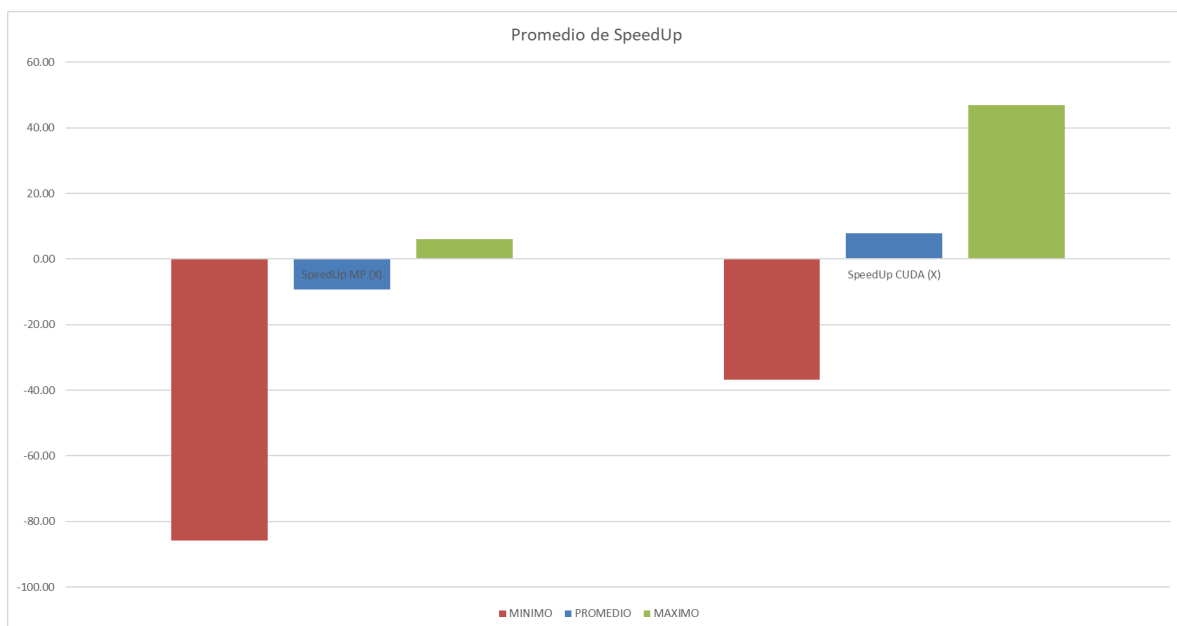
4.4. Comparativa Estrategias (21N)

En esta sección tomaremos el speedUp y las mejoras finales de las estrategias en paralelo en comparación con la versión secuencial, en un sistema de 21 nodos, lo suficientemente grande como para notar diferencias significativas.



Analizando primeramente la gráfica de tiempos, podemos apreciar como la versión **secuencial** es casi siempre la **más demorada**, a excepción de las pruebas en donde se reduce en gran medida el análisis de variables, lo que baja la complejidad del sistema a entre 10 y 15 variables (por eso con multiprocessing es más demorado). Podemos ver como la gráfica morada, que es la estrategia en paralelo con **multiprocessing reduce significativamente el tiempo** cuando el **sistema es grande**, pero aumenta en **cierta medida** estos tiempos cuando el **sistema es pequeño**.

La **más eficiente** finalmente sería **CUDA**, que aparte de **reducir el tiempo global en gran medida**, también es **eficiente en los grupos pequeños**. Esto se traduce en la siguiente gráfica de speedUp promedio:



Donde podemos apreciar que con Multiprocessing, el promedio del speedUp es negativo. mientras que con CUDA tenemos un promedio positivo y hasta un speedup máximo de más de 40x.

4.5. Comparativa Estrategias (21N) - Tiempos por función

COMPARACION TIEMPOS POR FUNCION EN 21N					
Proceso	Geometric (s)	Geometric MP (s)	Geometric MP CUDA (s)	SpeedUp MP (x)	SpeedUp CUDA (x)
1	0.35	0.33	0.27	1.06	1.30
2	829.35	148.22	4.28	5.60	193.77
3	14.05	20.03	13.32	0.70	1.05
4	0.23	22.95	23.72	0.01	0.01
5	843.98	191.53	41.59	4.41	20.29
1	Descomposicion en Tensores				
2	Calculo de la tabla de costos				
3	Identificacion de Candidatos + EXTRA + Filtrado				
4	Evaluacion de candidatos				
5	Tiempo Final				

Aquí podemos apreciar en primer lugar la enorme diferencia entre usar paralelización y no, sobre todo en la principal función que aporta tiempo a nuestro algoritmo como lo es el cálculo de la tabla de costos. Podemos apreciar cómo podemos lograr hasta un SpeedUp final de 194x cuando usamos CUDA lo que impacta enormemente en los algoritmos.

En el resto de funciones apreciamos como en la distribución general no aportan mucho, a comparación del cálculo de costos, a excepción de la identificación y evaluación de candidatos. Aquí hay un enorme detalle, es que dependiendo el sistema se pueden generar más o menos candidatos, cuando se generan muchos, se nota la paralelización, pero cuando son pocos como en este caso se puede apreciar como es mejor la versión secuencial (solo en este paso). pero a rasgos generales con CUDA tenemos una eficiencia mucho mayor (quitando este peor) que en secuencial o paralelo con Multiprocessing.

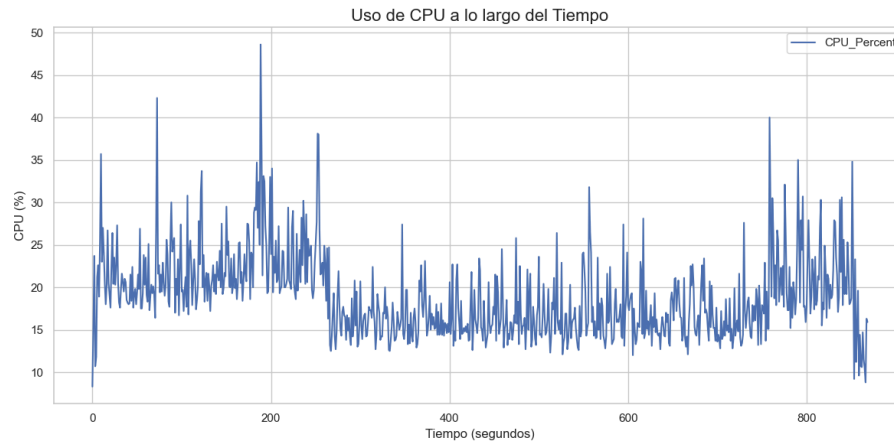
4.6. Uso de recursos del sistema

Durante la ejecución de cada estrategia de procesamiento, se monitoreó el uso de recursos del sistema (CPU, memoria RAM y, cuando corresponde, GPU) con el fin de evaluar el impacto computacional de cada una y su eficiencia en términos de aprovechamiento del hardware disponible. Todos los recursos monitoreados se hacen haciendo uso del *mayor subsistema probado (21N)* para identificar con mayor facilidad los cambios de tiempo y uso.

- **Geometric – versión secuencial**

En la versión original secuencial, el uso de CPU se mantiene en un rango moderado, oscilando principalmente entre el 15% y el 35%, con algunos picos

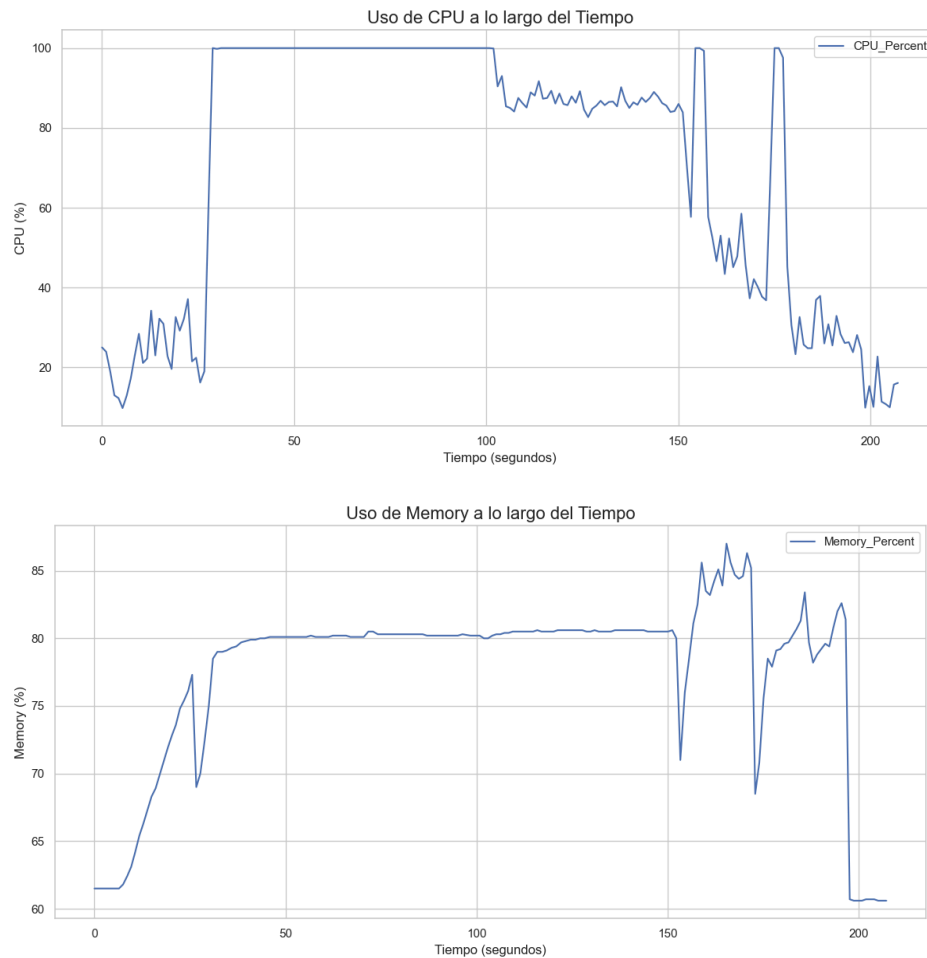
que superan el 40% (Ocasionados presumiblemente por programas de terceros del equipo de pruebas). Esta variabilidad refleja un procesamiento que no aprovecha completamente los recursos de cómputo disponibles, ya que solo se utiliza un núcleo. En cuanto a la memoria RAM, se observa una carga inicial fuerte cercana al 75%, seguida de un comportamiento relativamente estable alrededor del 63% durante el resto de la ejecución. Esto indica que, aunque el uso de CPU es limitado, la carga de datos sí representa un consumo sostenido de memoria. Finalmente se observa como al finalizar el consumo de RAM normal del pc es de cerca del 57.5%



- **Geometric + MP**

La estrategia que incorpora paralelización con multiprocessing muestra un comportamiento distinto. El uso de CPU se eleva rápidamente hasta el 100% durante la fase inicial y se mantiene alto por un periodo considerable (ya que la mayoría de funciones hacen uso de esta paralelización), lo que demuestra que la carga de trabajo se distribuye eficientemente entre múltiples núcleos del procesador. Sin embargo, el uso de memoria también incrementa notablemente, alcanzando valores superiores al 85%. Este comportamiento es coherente con la

replicación de datos en múltiples procesos y la necesidad de mantener estructuras en paralelo, lo cual introduce una sobrecarga que puede volverse crítica si no se gestiona adecuadamente (por ejemplo disminuir el número de procesos en redes más grandes). Además, se observan caídas abruptas tanto en CPU como en memoria cerca del final, lo que podría indicar etapas de sincronización o liberación de recursos. Finalmente se recomienda en sistemas menores a 20N hacer uso de las funciones secuenciales para identificar candidatos en vez de las versiones paralelas para una mayor mejora, o en su defecto usar directamente la versión secuencial completa (que es más óptima para estos casos).

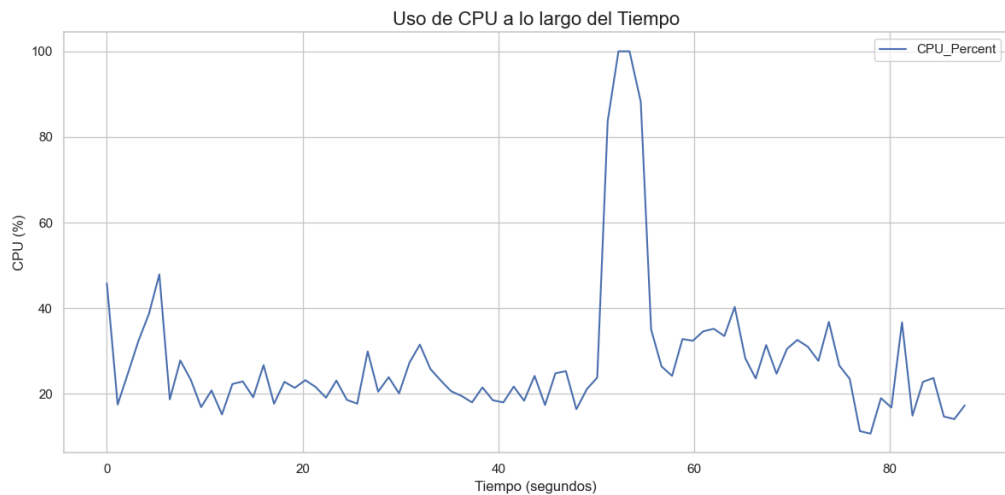


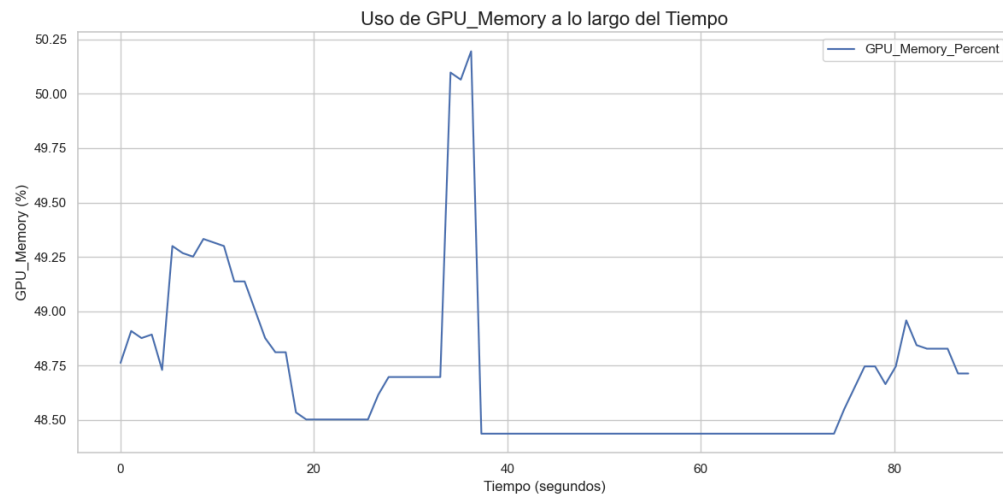
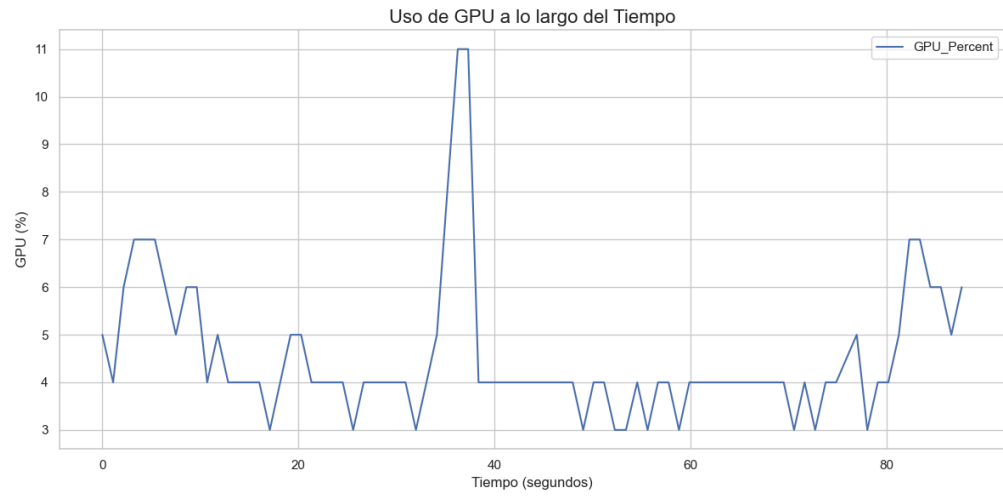
- Geometric + MP + CUDA**

La versión que combina multiprocessing con ejecución en GPU mediante CUDA presenta un patrón de uso más equilibrado. El uso de CPU es menor en comparación con la versión puramente en multiprocessing ya que solo se hace uso de esta herramienta en la parte final de calcular costos de biparticiones candidatas, lo que sugiere una correcta delegación de tareas a la GPU. El uso de

GPU alcanza picos de hasta un 11%, y su memoria asociada (VRAM) llega a valores cercanos al 50.25%, lo que indica una carga significativa de trabajo en la tarjeta gráfica. En paralelo, la memoria RAM del sistema también se incrementa progresivamente, con varios picos por encima del 75%, reflejando la combinación de carga en CPU, GPU y memoria principal. A pesar de este uso intensivo de recursos, el tiempo de ejecución global es considerablemente menor, y los recursos se liberan de forma más controlada.

Finalmente destacar, que aunque no se hace uso al 100% de la GPU, se nota una diferencia significativa en el cálculo de la tabla de costos en comparación con multiprocessing, logrando bajar en 21 nodos, el tiempo de este cálculo de 150s hasta 5s.





5. Conclusiones

Finalmente podemos concluir después de realizar todas estas pruebas que si se desean analizar subsistemas que tengan una cantidad de variables **inferior a 20**, **GEOMETRIC** es la mejor opción ya que los costos de sincronización entre varios núcleos de CPU o GPU no afectarán el rendimiento. Si se desea usar la versión con Multiprocessing estos costos de sincronización afectarán todo el rendimiento y darán el peor resultado. En comparación con CUDA que estos costos de sincronización son menores y podría sacar un rendimiento mejor (aunque inferior a la versión secuencial).

Para **más de 20 Nodos** se aconseja usar **GEOMETRIC + MP + CUDA**, que es la estrategia más optimizada tanto en términos de tiempo, como uso de los recursos disponibles.

Finalmente, tomar en cuenta la cantidad de variables que estén “**activas**” tanto en el **alcance** como en el **mecanismo**. Ya que esto reduce el tamaño de la tabla de costos y en consecuencia en algunos casos es como si solo analizáramos “10 variables” por lo que el rendimiento paralelo será peor.